
Can This Agent Even Do That?

A Decidable Goal-Achievability Type Discipline for LLM-Synthesized Agent Skills

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Autonomous agents are increasingly configured by *skills*: natural-language spec-
2 ifications that an LLM drafts from a user’s stated intent and then executes, often
3 across several cooperating agents. Such drafts fail expensively at run time on
4 plans that were impossible from the start — a step that invokes a tool the agent
5 does not have, a goal conjunct nothing can establish, a budget no run can meet,
6 or a multi-agent handoff that deadlocks. We present a *skill compiler* that decides,
7 before execution and without an LLM judge, whether a skill’s *goal is achievable*
8 *at all*. The compiler fuses two mature disciplines not previously combined for this
9 purpose: capability-guarded reachability from automated planning, and deadlock-
10 freedom from multiparty session types (MPST), decided *directly* over session con-
11 figurations and a goal-marked global type synthesized from intent, following the
12 synthetic MPST discipline of checking a session directly against a global pro-
13 tocol [7, 8]. The LLM that compacts prose into a formal pack of preconditions,
14 effects, and goals is treated as *untrusted*; the checker is specified by a Coq-verified
15 refutation-sound abstraction theorem. Relative to the declared pack, an *impossi-*
16 *ble* verdict is never wrong; whether the pack is faithful to the prose is a separately
17 deferred obligation. The positive verdict remains deliberately incomplete, with
18 payload faithfulness and intent fidelity handed respectively to a runtime monitor
19 and a human checkpoint. We also establish subject reduction and session fidelity
20 for the direct discipline: the single-label communication fragment with bystander
21 interleaving is mechanized, while world-changing action interleaving is proved on
22 paper under explicit goal-stability and effect-commutativity hypotheses. We give
23 the syntax, labelled operational semantics, typing and achievability judgments,
24 and the metatheory: refutation, tolerance and capability-monotonicity soundness;
25 operational correspondence; decidability within a static-topology fragment; and
26 an undecidability boundary for an extension with dynamic participant spawning.
27 An implementation and reproducible corpus exercise the canonical failure modes,
28 the fragment boundary, and the two deferred edges.

29 1 Introduction

30 A modern agent is rarely a single model call. It is a *skill*: a specification — increasingly authored
31 in natural language and drafted by an LLM from a user’s stated goal — that names tools, describes
32 a plan, and coordinates one or more agents toward an outcome. The appeal is that a non-expert can
33 say *what* they want and have the system assemble the *how*. The hazard is that the assembled how
34 is frequently *impossible*, and nothing notices until execution has already spent tokens, taken actions,
35 and, in the multi-agent case, deadlocked halfway.

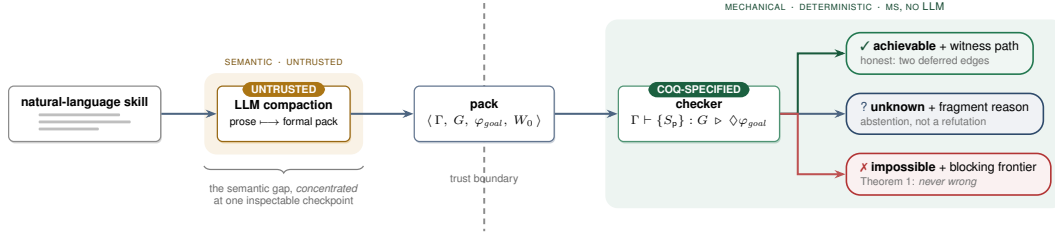


Figure 1: The trust boundary. An untrusted LLM compacts prose into a formal pack; a deterministic checker governed by a Coq-verified abstraction theorem decides achievability over the pack. The verdicts are asymmetric: *impossible* is sound (Theorem 1), *achievable* is honest about its deferred obligations (Section 3.1).

36 The failures are structural and recurring. A plan *books a flight and emails a confirmation* although no
 37 email tool is in scope (a hallucinated capability). A plan pursues *a flight under \$500* when the only
 38 booking tool returns premium fares (an unsatisfiable refinement). A planner waits for a worker's
 39 result while the worker waits for the planner's answer, each listening for a message the other never
 40 sends (a coordination deadlock). In every case the goal was unreachable *a priori*, and a cheap static
 41 check could have said so.

42 This paper asks a deliberately narrow question and answers it with a guarantee: **given an LLM-**
 43 **drafted skill, can its goal be achieved at all?** We are not (here) verifying that every step is safe,
 44 nor that the agent behaves correctly on every input. We decide *achievability*, tolerantly — small
 45 details, detours, and payload variation must not raise a false alarm — and *soundly for refutation*:
 46 when the compiler says a goal is impossible, it is right.

47 1.1 The idea, and the trust boundary

48 The architecture has two halves separated by a trust boundary. An LLM performs *compaction*: it
 49 reads the prose skill and distills it into a formal *pack* — capabilities annotated with preconditions
 50 and effects, a goal-marked global protocol, and an initial state. This step is *untrusted*; the LLM may
 51 misread the prose. A deterministic *checker* then decides achievability over the pack, and the checker
 52 is mechanically proved sound. The division is what makes the system honest: a faulty compaction
 53 can only ever cause a false *achievable* (caught downstream), never a false *impossible* about the pack
 54 it received (Figure 1).

55 Crucially, the semantic gap between *what the user meant* and *what was formalized* is not eliminated
 56 — it is *relocated and concentrated* into one inspectable checkpoint at compaction time, rather than
 57 scattered, unobserved, across every run. That relocation is the central design move.

58 1.2 Contributions

- 59 1. **A goal-achievability type discipline** (Section 3 and Appendices A–C) that fuses capability-
 60 guarded reachability (planning) with deadlock-freedom (multiparty session types) over a
 61 goal-marked global type synthesized from intent. Following synthetic MPST [7, 8], the
 62 declarative discipline types whole session configurations directly; our contribution is to add
 63 the guarded world, goal observation, and refutation-sound/achievement-incomplete verifi-
 64 cation asymmetry for LLM-drafted agent skills.
- 65 2. **A mechanized soundness specification** (Appendix D) in Coq: a refutation-sound ab-
 66 straction theorem parameterized by simulation hypotheses, tolerance soundness (coarsen-
 67 ing the abstraction preserves refutation), and capability monotonicity (more tools never
 68 make a goal impossible) — all axiom-free — together with an *operational-correspondence*
 69 metatheory (subject reduction and session fidelity) for the direct judgment over a labelled
 70 transition system. Full bystander interleaving is mechanized for single-label communica-
 71 tion; the world-changing action cases are paper proofs under explicit side conditions.
- 72 3. **A decidability boundary result** (Appendix D): achievability is decidable in a static-
 73 topology fragment; an explicit extension with unbounded participant spawning crosses to
 74 undecidability.

75 4. **An implementation and evaluation** (Sections 4–5): a checker, an LLM-compaction front-
 76 end with a schema gate, and a corpus spanning the canonical failure modes. Results match
 77 the theory: zero false impossibilities; every false achievability is one of the two deferred
 78 residues.

79 2 Overview by Example

80 Consider a one-agent skill whose goal is *a flight is booked and a confirmation is sent*. The author
 81 writes prose; the LLM compacts it. Two compactions differ only in whether an email tool is in
 82 scope.

$$\varphi_{goal} = \text{booked} \wedge \text{confirmation_sent}$$

83	variant A (no email tool) search : <i>Add</i> {searched} filter : <i>pre</i> searched; <i>Add</i> {filtered} book : <i>pre</i> filtered; <i>Add</i> {booked}	variant B ... A, plus: email : <i>pre</i> booked; <i>Add</i> {confirmation_sent}
----	--	--

84 Under STRIPS *frame semantics* — a predicate is false unless an effect makes it true — variant A has
 85 no action establishing `confirmation_sent`, so no reachable state satisfies the goal: the checker returns
 86 *impossible* and points at the missing establisher. Variant B reaches the goal along `search · filter ·`
 87 `book · email`, returned as a witness. The same example is mechanized in Appendix D (the concrete
 88 instance `FlightInstance`): variant A is *provably* unachievable, yet a booking is still reachable —
 89 the refutation is about the missing tool, not a dead protocol.

90 Tolerance is essential, and is where a naive reachability check would misfire. Extra status messages,
 91 a clarifying detour, or payload variation must not refute the goal. We obtain this from three or-
 92 thogonal relaxations made precise in Appendices A–C: *existential may-reachability* (a single good
 93 path suffices), *session subtyping* (interface slack in the safe direction, folded directly into the sin-
 94 gle conformance rule rather than a separate relation), and *goal-relevant abstraction* (payload detail
 95 collapsed before checking).

96 3 Verification Model and Signals

97 **Declared pack.** The checker consumes $P = \langle \Gamma, G, \varphi_{goal}, W_0 \rangle$: capability declarations Γ with
 98 guards and STRIPS-style effects; a goal-marked global protocol G (formally, a *global type*; Ap-
 99 pendix A); a logical goal φ_{goal} ; and an initial abstract world W_0 . The LLM that compacts prose
 100 into P is outside the trusted core. A deterministic schema gate first enforces the decidable frag-
 101 ment: finite static roles, guarded recursion, finite-domain booleans, and quantifier-free linear integer
 102 arithmetic. The formal syntax and labelled semantics are given in Appendices A–B.

103 **Heterogeneous verification signals.** Achievability is not reduced to a learned score. The judg-
 104 ment composes signals that live on different substrates:

Signal	Formal role	Evidence returned
tool availability and effects coordination structure	capability guards; world transition direct global conformance	missing capability or witness action offending handoff or conformant proto- col
goal and cost refinements	logical reachability; QF-LIA	unsatisfied conjunct, blocked guard, or model
human intent review runtime payload checks	validates prose-to-pack fidelity deferred monitor obligation	accepted or revised declared goal blame-bearing execution event

106 The first three signals are composed before execution. The latter two are explicit obligations rather
 107 than assumptions silently folded into the static verdict.

108 **Decision procedure.** After the schema gate, the checker (i) rejects references to undeclared capa-
 109 bilities and goal atoms with no establisher; (ii) checks declared role behaviors against G , requiring
 110 exact sender choices while allowing receiver-side branch supersets; and (iii) explores existential pro-
 111 tocol/world successors from W_0 , using Z3 for guards and refinements. An unsatisfied goal marker

112 is a blocking checkpoint. Solver timeouts and packs outside the static fragment cannot justify refu-
 113 tation. The interface is therefore tri-valued:

IMPOSSIBLE(F)	a sound blocking frontier F ,
ACHIEVABLE(π, O)	a witness path π and deferred obligations O ,
UNKNOWN(r)	an abstention with fragment reason r .

114 Verdicts also carry the pack digest, checker semantics, and artifact version so that feedback remains
 115 attributable to the exact declaration checked.

116 **Guarantees.** Let *abs* map concrete protocol/world configurations to abstract configurations, and
 117 assume step and goal simulation for the declared pack (Appendix D). If no abstract run reaches
 118 φ_{goal} , no concrete run represented by that pack reaches the concrete goal (Theorem 1). Coarsening
 119 α and adding capabilities cannot create a false refutation (Theorems 2–3). Direct conformance has
 120 subject-reduction and session-fidelity results (Theorems 4–5); the Coq artifact covers single-label
 121 communication with bystander interleaving, while the paper states explicit stability and commuta-
 122 tivity assumptions for world-changing interleavings. The static fragment is decidable; unbounded
 123 dynamic spawning is an explicit undecidable extension, for which the implementation abstains. Full
 124 rules, assumptions, proofs, and mechanization scope appear in Appendix D.

125 3.1 Structured feedback and human oversight

126 The blocking frontier and witness are inspectable, heterogeneous feedback rather than a scalar re-
 127 ward. A bounded repair adapter may ask the untrusted LLM to fix a non-projectable protocol, but
 128 it may neither invent capabilities nor weaken the goal, and every revision is rechecked determinis-
 129 tically. This realizes a small agent-verifies-agent loop while keeping the human checkpoint at the
 130 irreducible intent-fidelity stage. Automated prompt or scaffold search is an application of these
 131 verdicts, not an empirical contribution claimed here.

132 4 Implementation

133 The executable decision path is approximately 1.35K lines of Python, including the checker, pack
 134 gate, formula layer, and session adapter. The compaction front-end issues the artifact prompt
 135 to an LLM and passes the result through a deterministic *schema gate* before the checker sees
 136 it, so malformed packs are rejected without crashing the trusted core. The Coq artifact contains
 137 three developments and two assumption-audit harnesses: reachability soundness core (Theorems 1–
 138 3 plus `FlightInstance`, ~230 lines); the direct-typing core (the head-move action safety plus
 139 `HandoffInstance`, ~310 lines); and the operational-correspondence core (Theorems 4–5 for the
 140 single-label communication fragment with full interleaving, ~250 lines); all three are checked in
 141 CI under pinned Coq 8.18.0 with no axioms, verified by `Print Assumptions`. The checker im-
 142 plements the corresponding abstractions from Appendices A–C: STRIPS frame semantics for the
 143 world, existential search over the protocol control structure, and Z3 for guard/goal satisfiability and
 144 arithmetic refinements. On success it returns the witness path; on refutation, the blocking frontier
 145 (missing capability, unsatisfiable guard, unestablished goal conjunct, or non-projectable handoff).
 146 The optional LLM front-end may use a `NON_PROJECTABLE` counterexample for one bounded,
 147 structure-only repair round; it may not invent capabilities or weaken the goal, and the determin-
 148 istic checker remains the final referee. This is a small reflective-verification loop, not a general
 149 scaffold-search result.

150 The conformance step ($G \vdash (\mathbb{M}; W)$, Section C.2) is approximated *algorithmically* in `session.py`.
 151 The adapter first derives the behavior expected of each role (projection), reconciles choices that
 152 a bystander cannot observe (merge), and then compares each declared role with that expectation
 153 (direct conformance). The declarative judgment is what the checker must decide, projection-with-
 154 merge remains a sound and, we expect, complete decision procedure for it whenever G is realizable
 155 — that equivalence is asserted from the classical correspondence between projection and network-
 156 level conformance [1, 4], not itself mechanized here. The conformance adapter matches T-COMM:
 157 sender labels must equal the protocol labels, while a receiver may accept a superset. A separate
 158 generic Gay–Hole subtyping utility remains in the artifact but does not decide this premise. The
 159 verdict reason `NON_PROJECTABLE` names this algorithmic engine’s diagnostic; Section C.2 is the
 160 specification it is checked against.

5 Evaluation

We assembled a corpus of 15 skills spanning the canonical failure modes — hallucinated planning, retry-forever loops, coordination deadlock, refinement violations — with achievable controls (including detours and informed branches) and two deliberately *spurious* cases exercising the deferred edges. Each skill carries a ground-truth semantic label; the positive class is *achievable*. Evaluation is deterministic and CPU-only: it requires no training or live model inference, completes the 15-case matrix in under one second on a commodity laptop, and gives each individual Z3 query a 10-second timeout.

	predicted achievable	predicted impossible
truly achievable	TP = 6	FN = 0
truly impossible	FP = 2	TN = 7

Two claims, both confirmed. **Soundness:** FN = 0 — no achievable goal was wrongly refuted, the empirical face of Theorem 1 (achievability recall $6/6 = 100\%$). **Incompleteness, contained:** FP = 2, and both are exactly the planted spurious cases — one a false payload post-condition (Layer-C obligation), one an under-specified goal (intent edge). No structural failure was missed in this proof-of-concept corpus; this is evidence of coverage, not a proof that the two residues exhaust all empirical failure modes. Each refutation reason fires on its matching mode:

Failure mode (source)	Verdict reason
hallucinated planning — invokes a non-existent tool	MISSING_CAPABILITY
hallucinated planning — no establisher for a goal conjunct	GOAL_UNSAT
retry-forever loop — guard never satisfiable	BLOCKED_GUARD
coordination deadlock — unobserved choice / bad handoff	NON_PROJECTABLE
refinement violation — budget unsatisfiable on every run	GOAL_UNSAT

A separate six-case fragment suite exercises behavior absent from the headline matrix: tail-recursive retry and non-terminating control, dynamic spawning, protocol-independent refutation despite spawning, and conformant versus non-conformant declared role behavior. It yields two ACHIEVABLE, three IMPOSSIBLE, and one UNKNOWN. The abstention is reported separately rather than counted as a false negative, because UNKNOWN makes no refutation claim. Together the suites contain 21 declared packs; the 15-case matrix remains the only binary ground-truth matrix.

6 Related Work

Prior-art note. The claims below reflect the authors’ survey at submission time; the fusion’s novelty should be confirmed against a systematic search of the planning-meets-behavioural-types literature, which is dispersed across communities.

Multiparty session types [1, 2, 4] supply deadlock-freedom via projection and subtyping [3, 5]; our calculus and directed-choice global types follow the modern synchronous formulation of the Yoshida line [4–6]. We adopt the synthetic line [7, 8]: Section C.2 checks conformance directly against a whole session configuration rather than projected local types. Our direct rule corresponds to the conservative, plain-merge condition that bystanders behave uniformly across an unobserved choice; it does not subsume every protocol accepted by modern full merge. The new contribution is the world and goal layer: capability guards, effects, goal observation, and the asymmetric refutation guarantee are not modeled by classical or synthetic MPST. *Automated planning* [9] supplies capability-guarded goal reachability; we reuse its frame semantics but embed it in a multiparty choreography. The contribution is the *fusion*, decided over an goal-marked global type synthesized by an LLM. Recent agent-verification work is adjacent but differently aimed: *provable coordination via message sequence charts* [11] projects a global choreography for LLM agents but targets coordination correctness, not goal achievability; its runtime-planning extension also establishes that LLM synthesis of a coordination protocol is itself prior art. *VeriGuard* [12] separates offline policy verification from online monitoring as a safety shield; *agent behavioural contracts* [13] provide probabilistic, runtime-enforced compliance and composition conditions for multi-agent chains, whereas our verdict is static and sound for refutation relative to the pack.

Skill-bundle scanners and verified-skill catalogs are complementary. NVIDIA’s SkillSpector [14] and Verified Agent Skills pipeline [15] address the same deployment object—portable agent skills—but ask a different question. SkillSpector is a pre-publication security control: it normalizes a skill bundle and runs deterministic scanners such as static pattern detectors, AST and taint analyzers, manifest-consistency checks, metadata-poisoning checks, supply-chain checks, and optional LLM-backed semantic analyzers. This is valuable admission control for skills, but it is not a compiler or type discipline in the sense used here. As published, SkillSpector does not aim to provide an operational semantics of goal-marked skills, a goal-reachability decision, or a refutation-soundness theorem. A SkillSpector-like pass belongs at the boundary of the compiler: before or alongside LLM compaction, it can vet the incoming SKILL.md, code, dependencies, and MCP metadata; after compaction, it can help justify the honesty of declared capabilities and least-privilege metadata. The type checker then consumes the sanitized formal pack and proves the narrower claim that the declared goal is not structurally impossible.

None of the above frames the static check as goal achievability over a declared world and goal-marked protocol, nor gives the same refutation-sound/achievement-incomplete asymmetry.

7 Limitations and Future Work

- **Relative soundness.** Everything is proved about the declared Γ and S . If the prose lies (a “read-only” tool that writes), the checker verifies a fiction; honest-declaration is a separate obligation, of which this makes only the interaction slice decidable. A practical compiler should therefore include a skill-bundle security pass—for example a SkillSpector-like scanner—to inspect SKILL.md, scripts, dependencies, manifests, and permission metadata before the formal pack is trusted by the achievability checker.
- **Static topology for totality.** Dynamic spawning drops the procedure to a semi-decision (Proposition 2); a practical system answers UNKNOWN rather than guessing.
- **Trusted abstraction.** A coarse α silently widens tolerance (more false achievable), never breaking Theorem 1 but weakening usefulness.
- **Engineering gaps.** The mechanization covers the generic refutation-sound abstraction theorem and concrete examples, head-move action safety, and subject reduction/session fidelity with full bystander interleaving for the single-label communication model, all axiom-free. The executable checker is schema-gated and tested against that specification, but the STEP-SIM/GOAL-SIM instantiation for its exact symbolic state is not mechanized. The Coq correspondence model also does not yet combine observable goal labels, participant matching, multi-branch choice, and world-changing interleaving in one development. Still open are that faithful mechanization; a head-normalization/permutation bridge from interleaved session runs to the head-ordered reachability search; equivalence between the declarative judgment and the projection-based adapter; and a concrete-session realization theorem for abstract witnesses. The 21-pack evaluation remains a proof of concept.

Broader impact. Pre-execution refutation can reduce wasted computation and prevent structurally impossible agent plans from reaching deployment. The same formal verdict can, however, create false confidence when an inaccurate pack omits a harmful tool effect or weakens the user’s actual goal. We therefore expose the declared pack, witness or blocking frontier, and deferred obligations rather than presenting a single trust score. Intent review and runtime payload monitoring remain necessary before the method is used in consequential settings; this work does not evaluate fairness, privacy, or deployment safety.

8 Conclusion

An LLM can draft an agent’s plan from intent; the open question has been whether anything can tell, cheaply and before execution, that the plan is doomed. For the *achievability* question the answer is yes, relative to a declared pack: combine planning’s capability-guarded reachability with a synthetic, directly-typed global protocol; treat LLM compaction as untrusted; and use a refutation-sound abstraction so an *impossible* verdict is never wrong about that declared model. *Achievable* remains honest about payload and intent fidelity, while *Unknown* abstains outside static topology.

This yields a structured, inspectable verification signal for agent skills and a path toward, rather than a completed proof of, stronger per-step safety.

References

- [1] K. Honda, N. Yoshida, M. Carbone. Multiparty Asynchronous Session Types. *J. ACM* 63(1), 2016.
- [2] E. Li, F. Stutz, T. Wies, D. Zufferey. Complete Multiparty Session Type Projection with Automata. *CAV* 2023.
- [3] S. Gay, M. Hole. Subtyping for Session Types in the Pi Calculus. *Acta Informatica* 42(2–3), 2005.
- [4] N. Yoshida, L. Gheri. A Very Gentle Introduction to Multiparty Session Types. *ICDCIT, LNCS* 11969, 2020.
- [5] S. Ghilezan, S. Jakšić, J. Pantović, A. Scalas, N. Yoshida. Precise Subtyping for Synchronous Multiparty Sessions. *J. Log. Algebr. Meth. Program.* 104, 2019.
- [6] A. Scalas, N. Yoshida. Less is More: Multiparty Session Types Revisited. *Proc. ACM Program. Lang.* 3(POPL), 2019.
- [7] S.-S. Jongmans, F. Ferreira. Synthetic Behavioural Typing: Sound, Regular Multiparty Sessions via Implicit Local Types. *ECOOP, LIPIcs* 263, 2023.
- [8] D. Castro-Perez, F. Ferreira, S.-S. Jongmans. A Synthetic Reconstruction of Multiparty Session Types. *POPL*, 2026.
- [9] R. Fikes, N. Nilsson. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. *Artif. Intell.* 2(3–4), 1971.
- [10] D. Brand, P. Zafiropulo. On Communicating Finite-State Machines. *J. ACM* 30(2), 1983.
- [11] B. Bollig, M. Függer, T. Nowak. Provable Coordination for LLM Agents via Message Sequence Charts. *ISoLA*, 2026. arXiv:2604.17612.
- [12] L. Miculicich, M. Parmar, H. Palangi, K. D. Dvijotham, M. Montanari, T. Pfister, L. T. Le. VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation. arXiv:2510.05156, 2025.
- [13] V. P. Bhardwaj. Agent Behavioral Contracts: Formal Specification and Runtime Enforcement for Reliable Autonomous AI Agents. arXiv:2602.22302, 2026.
- [14] N. Paz, K. Pradeep, N. Raghavan, A. Nikirk, M. Gupta. SkillSpector: A Pre-Publication Security Control for Agent Skills. *Agent Skills Workshop at CAIS*, 2026.
- [15] NVIDIA. NVIDIA-Verified Agent Skills Provide Capability Governance for AI Agents. NVIDIA Technical Blog, 19 May 2026.

A Syntax

We fix disjoint sets of predicates $p \in \text{Pred}$ (boolean), numeric variables $x \in \text{Var}$, roles $p, q, r \in \mathcal{R}$, capability names $a \in \text{Cap}$, and message labels $\ell \in \text{Lab}$. Following the modern presentation of multiparty session types [4–6], we proceed bottom-up: state and capabilities (Sections A.1–A.2), then *processes* — the programs skills execute as — (Section A.3), then the global *type* they are checked against directly (Section A.4). In each grammar, “ $::=$ ” lists the possible forms of an expression. In each inference rule below, the statement beneath the line follows when every premise above the line holds. Rule names identify the corresponding checker obligation; they are not additional assumptions.

A.1 State logic

$$\begin{aligned} e &::= x \mid n \mid e + e \mid e - e \mid c \cdot e & (c, n \in \mathbb{Z}) \\ \varphi &::= p \mid \top \mid \perp \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid e \bowtie e & \bowtie \in \{<, \leq, =, \geq, >, \neq\} \end{aligned}$$

The state logic $\mathcal{L}$ is quantifier-free linear integer arithmetic with uninterpreted boolean predicates — a decidable fragment for which entailment $\models$ and satisfiability are discharged by a satisfiability-modulo-theories (SMT) solver.

298 A.2 Worlds and capabilities

299 A world $W = \langle B, N \rangle$ assigns truth values to predicates $B : \text{Pred} \rightarrow \mathbb{B}$ and numeric variables
300 $N : \text{Var} \rightarrow \mathbb{Z}$. A *capability context* Γ maps each available tool a to a guarded effect:

$$\begin{aligned} \Gamma(a) &= \langle pre_a, eff_a \rangle, \quad pre_a \in \mathcal{L}, \\ eff_a &= \langle Add_a \subseteq \text{Pred}, Del_a \subseteq \text{Pred}, Asg_a : \text{Var} \rightarrow e, Nd_a : \text{Var} \rightarrow \mathcal{L} \rangle \end{aligned}$$

301 $a \notin \text{dom}(\Gamma) \iff$ the agent does *not* possess tool a (no hallucinated tools).

302 *Add/Del* are the STRIPS add- and delete-lists; *Asg* gives deterministic numeric updates $x := e$;
303 *Nd* gives *nondeterministic* updates $x := *$ constrained by a formula over the new value (used to
304 model post-conditions such as “books a fare under 500”).

305 A.3 Processes and multiparty sessions

306 Skills *execute* as processes; a multi-agent run is a session of named processes. We use the syn-
307 chronous multiparty session calculus in its modern coinductive presentation [4, 5]: terms are *regular*
308 trees (finitely many distinct subtrees; $\mu X.P$ is notation for a cycle, identified with its unfolding), and
309 labels are already goal-relevant tokens, $\ell = \alpha(m)$ for concrete messages m — the tolerance dial α
310 is a parameter of the label alphabet.

$$P ::=^{coind} \mathbf{0} \mid p!\{\ell_i.P_i\}_{i \in I} \mid p?\{\ell_i.P_i\}_{i \in I} \mid a.P \quad \mathbb{M} = p_1[P_1] \parallel \cdots \parallel p_n[P_n]$$

311 with I finite and non-empty, labels ℓ_i pairwise distinct, and session names pairwise distinct. The
312 output $p!\{\ell_i.P_i\}$ *internally* chooses a label and sends it to p ; the input $p?\{\ell_i.P_i\}$ offers an *external*
313 choice; $a.P$ invokes capability a — the only construct that changes the world. A skill S_p is a process:
314 the behaviour declared for role p .

315 A.4 Global types

316 The single communication construct of a global type is a *directed* labelled choice — selection by p
317 and branching at q in one construct. (A partnerless “ p selects” admits no coherent view for the roles
318 that did not choose; directed choice is the standard MPST construct [1, 4].)

$$G ::=^{coind} p \rightarrow q : \{\ell_i.G_i\}_{i \in I} \mid a@p.G \mid \checkmark\varphi.G \mid \text{End}$$

319 subject to $p \neq q$ and the same regularity and label conditions as processes. This is the *only* type in the
320 discipline: Section C.2 types a whole session directly against G — there is no local-type grammar,
321 no projection function, and no separate subtyping relation. (A local-type projection of G remains
322 a legitimate, more efficient *algorithmic* decision procedure for the same conformance judgment,
323 and is what our reference implementation runs, Section 4; Section C.2 gives the specification it
324 implements.) Goal markers $\checkmark\varphi$ live *only* in G : achievability is decided on the global type (rule
325 ACH), and markers are consumed at typing time by T-GOAL without ever burdening a process.
326 Only $a@p$ changes the world; messages carry coordination information only.

327 B Operational Semantics

328 B.1 World transitions

A capability fires when its guard holds, producing a successor world. Because *Nd* is nondeterminis-
tic, $\langle W \rangle a \langle W' \rangle$ may relate one world to many. The relation is implicitly indexed by Γ : the notation
 pre_a, eff_a is defined only when $\Gamma(a) = \langle pre_a, eff_a \rangle$, so every firing presupposes $a \in \text{dom}(\Gamma)$.

$$\begin{array}{c} \text{WORLD-ACT} \\ \hline W \models pre_a \quad W' \in \text{apply}(eff_a, W) \\ \hline \langle W \rangle a \langle W' \rangle \end{array}$$

329 $\text{apply}(eff_a, \langle B, N \rangle) = \langle B', N' \rangle, \quad B' = (B \cup Add_a) \setminus Del_a$

$$330 \quad N'(x) = \begin{cases} \llbracket Asg_a(x) \rrbracket_N & \text{if } x \in \text{dom}(Asg_a) \\ v \text{ with } \langle B, N[x \mapsto v] \rangle \models Nd_a(x) & \text{if } x \in \text{dom}(Nd_a) \\ N(x) & \text{otherwise (frame)} \end{cases}$$

331 B.2 A labelled semantics for sessions and global types

332 Only capabilities touch the world, so sessions reduce as configurations $(\mathbb{M}; W)$; communication is
 333 synchronous [5]. Each transition carries a label Λ recording its *participants*, so that independent
 334 moves by disjoint roles can be identified. Two auxiliary maps drive the labelled system: the *partic-*
 335 *ipants* $\text{prt}(\cdot)$ of a global type, a label, or a session, and the *capabilities* $\text{cap}(\cdot)$ (the set of moves a
 336 global type can perform). On global types,

$$\begin{aligned} \text{prt}(p \rightarrow q : \{\ell_i. G_i\}_{i \in I}) &= \{p, q\} \cup \bigcup_{i \in I} \text{prt}(G_i) & \text{prt}(\check{\varphi}. G) &= \text{prt}(G) \\ \text{prt}(a @ p. G) &= \{p\} \cup \text{prt}(G) & \text{prt}(\text{End}) &= \emptyset, \end{aligned}$$

337 on labels $\text{prt}(p \ell q) = \{p, q\}$, $\text{prt}(a @ p) = \{p\}$, $\text{prt}(\check{\varphi}) = \emptyset$, and on sessions $\text{prt}(\mathbb{M}) = \{p \mid \mathbb{M} \equiv$
 338 $p[P] \ \& \ P \neq \mathbf{0}\}$ (the roles with a residual behaviour). The capabilities of a global type are

$$\begin{aligned} \text{cap}(p \rightarrow q : \{\ell_i. G_i\}_{i \in I}) &= \{p \ell_i q \mid i \in I\} \cup \bigcup_{i \in I} \text{cap}(G_i) \\ \text{cap}(a @ p. G) &= \{a @ p\} \cup \text{cap}(G) & \text{cap}(\check{\varphi}. G) &= \{\check{\varphi}\} \cup \text{cap}(G) & \text{cap}(\text{End}) &= \emptyset. \end{aligned}$$

339 The capability names used by a protocol are

$$\text{caps}(G) = \{a \mid \exists p. a @ p \in \text{cap}(G)\}.$$

S-COMM

$$\frac{k \in I \subseteq J}{(p[q!\{\ell_i. P_i\}_{i \in I}] \parallel q[p?\{\ell_j. Q_j\}_{j \in J}] \parallel \mathbb{M}; W) \xrightarrow{p \ell_k q} (p[P_k] \parallel q[Q_k] \parallel \mathbb{M}; W)}$$

S-ACT

$$\frac{\langle W \rangle a \langle W' \rangle}{(p[a.P] \parallel \mathbb{M}; W) \xrightarrow{a @ p} (p[P] \parallel \mathbb{M}; W')}$$

S-GOAL

$$\frac{W \models \varphi}{(\mathbb{M}; W) \xrightarrow{\check{\varphi}} (\mathbb{M}; W)}$$

340 closed under permutation of $\parallel$ and $p[\mathbf{0}] \parallel \mathbb{M} \equiv \mathbb{M}$. A non-terminated configuration with no succes-
 341 sor is *stuck*; the deadlocking planner/worker handoff of Section 1 is the stuck two-role configuration
 342 in which both processes begin with an input. A goal marker is *not* carried by any process; instead S-
 343 GOAL lets a configuration *observe* that its world already entails one of its declared goals φ , emitting
 344 the label $\check{\varphi}$ without altering $\mathbb{M}$ or W . (Each session carries a set of goal formulas, and φ ranges
 345 over that set — not over every formula the world happens to satisfy — so that each observation
 346 has a matching marker in the protocol.) This observable goal step is matched at the type level by
 347 G-GOAL-E below, so that $\check{\varphi}$ is a genuine label Λ of both transition systems and the operational
 348 correspondence of Section D.3 ranges over it uniformly.

The checker relates a session to its protocol through a *labelled* transition system on global types,
 $(G, W) \rightsquigarrow_{\Lambda} (G', W')$, built from *head* rules that consume the leading construct and *interleaving*
 rules that let a move by participants *disjoint* from the head commute inward:

G-COMM-E

$$\frac{k \in I}{(p \rightarrow q : \{\ell_i. G_i\}_{i \in I}, W) \rightsquigarrow_{p \ell_k q} (G_k, W)}$$

G-ACT-E

$$\frac{\langle W \rangle a \langle W' \rangle}{(a @ p. G, W) \rightsquigarrow_{a @ p} (G, W')}$$

G-GOAL-E

$$\frac{W \models \varphi}{(\check{\varphi}. G, W) \rightsquigarrow_{\check{\varphi}} (G, W)}$$

G-COMM-I

$$\frac{(G_i, W) \rightsquigarrow_{\Lambda} (G'_i, W') \ (\forall i \in I) \quad \Lambda \in \text{cap}(G_i) \quad \text{prt}(\Lambda) \cap \{p, q\} = \emptyset}{(p \rightarrow q : \{\ell_i. G_i\}_{i \in I}, W) \rightsquigarrow_{\Lambda} (p \rightarrow q : \{\ell_i. G'_i\}_{i \in I}, W')}$$

G-ACT-I

$$\frac{(G, W) \rightsquigarrow_{\Lambda} (G', W') \quad \Lambda \in \text{cap}(G) \quad \text{prt}(\Lambda) \cap \{p\} = \emptyset}{(a @ p. G, W) \rightsquigarrow_{\Lambda} (a @ p. G', W')}$$

G-GOAL-I

$$\frac{(G, W) \rightsquigarrow_{\Lambda} (G', W') \quad \Lambda \in \text{cap}(G)}{(\check{\varphi}. G, W) \rightsquigarrow_{\Lambda} (\check{\varphi}. G', W')}$$

G-GOAL-E discharges a goal marker when $W \models \varphi$, emitting the observable label $\checkmark \varphi$ that matches S-GOAL. In the head-only reduction used for achievability, an unsatisfied marker is a checkpoint and blocks the path. In the full labelled correspondence, G-GOAL-I lets a continuation step commute inward under a still-pending marker. When that step changes the world, the correspondence theorem uses the goal-stability hypothesis stated in Section D.3; removing that hypothesis is future work. The extraction (head) rules G-COMM-E/G-ACT-E/G-GOAL-E give the Λ -erased reduction $(G, W) \rightsquigarrow (G', W')$ that the achievability judgment below uses; the interleaving rules matter only for the operational correspondence of Section D.3, and their $\Lambda \in \text{cap}(G)$ premise confines a commuting move to a capability the residual protocol actually offers. G-COMM-E is legitimate because choice is directed — the branch is a communication both endpoints observe, and Section C.2 checks that every third party can follow. A well-formed pack uses its declared goal φ_{goal} at each goal marker. A configuration is *goal-satisfying* when control is at such a marker or at the terminus and the world entails that declared goal:

$$\begin{aligned} & \text{goal}_{\varphi_{\text{goal}}}(\checkmark \varphi_{\text{goal}}.G, W) \iff W \models \varphi_{\text{goal}}, \quad \text{goal}_{\varphi_{\text{goal}}}(\text{End}, W) \iff W \models \varphi_{\text{goal}}, \\ & \Gamma; G \models \Diamond_{\text{init}} \varphi_{\text{goal}} \iff \exists W_0 \models \text{init}. \exists (G', W'). (G, W_0) \rightsquigarrow^* (G', W') \wedge \text{goal}_{\varphi_{\text{goal}}}(G', W'). \end{aligned}$$

The diamond is *existential*: a single witnessing run suffices — the formal source of detour-tolerance.

B.3 Realizability and subtyping, without projection

The classical route to ruling out a deadlocking handoff is *projection*: compute each role’s local type $G \upharpoonright r$ by structural recursion on G , taking the *merge* of a choice’s branches at every role that neither sends nor receives, and declaring G *realizable* exactly when this partial function is defined everywhere — undefinedness of the merge is the static signature of a role forced to behave differently in branches it cannot distinguish. Session subtyping $\leq$ is then a second, separate coinductive relation on local types, letting a role’s actual behaviour offer more external choices and fewer internal ones than its projected contract demands [3, 5, 6].

Both devices exist to answer questions about the *network as a whole* — “can every role tell the branches apart it needs to?”, “may this implementation stand in for that contract?” — while type-checking only one role’s syntax at a time. Section C.2 answers the same two questions with a single judgment, *typing the whole configuration directly against G* : a choice node’s rule quantifies over every branch the protocol declares and requires the *same* residual configuration to check against each one. This is the conservative plain-merge condition, obtained without ever computing $\sqcap$ or a local type. The receiver-side subtyping direction (a receiver may accept a superset of the protocol’s labels) is folded into the same rule as a label-set inclusion $I \subseteq J$. Tolerance therefore still has the same three independent knobs from Section 2 — the existential $\Diamond$ (Section B.2), subtyping (now inside T-COMM, Section C.2), and the granularity of α — “flexible, tolerant of small details” is still the profile $\langle \text{coarse } \alpha, \Diamond, \text{linear } \mathcal{L} \rangle$; a later safety layer flips the same machinery to the universal $\square$ over permission predicates.

C The Achievability Type System

Achievability now factors into three premises, each separately decidable, combined by the top rule ACH: no hallucinated capability, direct conformance of the declared skills to G , and reachability of the goal.

C.1 Capability and goal typing

$$\begin{array}{c} \text{T-EFF} \\ \frac{\Gamma(a) = \langle \text{pre}, \text{eff} \rangle \quad \varphi \models \text{pre}}{\Gamma \vdash \{\varphi\} a \{\text{sp}(\varphi, \text{eff})\}} \end{array} \qquad \begin{array}{c} \text{T-MISS} \\ \frac{a \notin \text{dom}(\Gamma)}{\Gamma \vdash a \triangleright \mathbf{X}(\text{MISSING_CAPABILITY})} \end{array}$$

$\text{sp}(\varphi, \text{eff})$ is the strongest postcondition of the STRIPS effect. T-MISS fires on hallucinated planning: the plan names a tool the context does not grant, and achievability is refuted without any search.

392 C.2 Direct conformance typing

Skills are typed *directly* against the global protocol and the world, by the coinductive judgment $G \vdash (\mathbb{M}; W)$ — read “ G types session $\mathbb{M}$ starting from world W .” There is no local type, no projection, and no separate subtyping relation: the receiver-side subtyping direction and the unobserved-choice check are structural side conditions of one rule.

$$\begin{array}{c}
\text{T-END} \\
\hline
\text{End} \vdash (\mathbf{p}[0]; W)
\end{array}
\qquad
\begin{array}{c}
\text{T-ACT} \\
\hline
\frac{G \vdash (\mathbf{p}[P] \parallel \mathbb{M}; W') \quad \langle W \rangle a \langle W' \rangle \quad \text{prt}(G) \cup \{\mathbf{p}\} = \text{prt}(\mathbb{M}) \cup \{\mathbf{p}\}}{a@p. G \vdash (\mathbf{p}[a.P] \parallel \mathbb{M}; W)}
\end{array}$$

$$\begin{array}{c}
\text{T-GOAL} \\
\hline
\frac{G \vdash (\mathbb{M}; W) \quad W \models \varphi \quad \text{prt}(G) = \text{prt}(\mathbb{M})}{\checkmark \varphi. G \vdash (\mathbb{M}; W)}
\end{array}$$

$$\begin{array}{c}
\text{T-COMM} \\
\hline
\frac{G_i \vdash (\mathbf{p}[P_i] \parallel \mathbf{q}[Q_i] \parallel \mathbb{M}; W) \quad \forall i \in I \subseteq J \quad \text{prt}(\mathbf{p} \rightarrow \mathbf{q} : \{\ell_i.G_i\}_{i \in I}) = \text{prt}(\mathbb{M}) \cup \{\mathbf{p}, \mathbf{q}\}}{\mathbf{p} \rightarrow \mathbf{q} : \{\ell_i.G_i\}_{i \in I} \vdash (\mathbf{p}[\mathbf{q}!\{\ell_i.P_i\}_{i \in I}] \parallel \mathbf{q}[\mathbf{p}?\{\ell_j.Q_j\}_{j \in J}] \parallel \mathbb{M}; W)}
\end{array}$$

393 T-ACT pairs type and world in lockstep with WORLD-ACT: the unconsumed type $a@p.G$ sits with
394 the *pre*-effect world W and the residual G with the *post*-effect world W' — the same convention as
395 G-ACT-E (Section B.2). In T-COMM the sender $\mathbf{p}$ offers exactly the protocol’s branch set I , while
396 the receiver $\mathbf{q}$ may accept *more* ($I \subseteq J$) — the one subtyping direction the rule keeps (SUB-EXT,
397 safe interface slack at the receiver). Requiring *every* protocol branch ℓ_i to be checked against the
398 *same* bystanders $\mathbb{M}$ is the plain-merge form of the unobserved-choice condition, so a role forced to
399 behave differently across branches it cannot distinguish makes no derivation possible — the rule fails
400 closed, and deadlock-freedom falls out of T-COMM itself with no merge to compute (Theorem 4).
401 Each rule additionally carries a *participant-matching* side condition tying the protocol’s participants
402 to the session’s: $\text{prt}(\mathbf{p} \rightarrow \mathbf{q} : \{\ell_i.G_i\}) = \text{prt}(\mathbb{M}) \cup \{\mathbf{p}, \mathbf{q}\}$ in T-COMM, $\text{prt}(G) \cup \{\mathbf{p}\} = \text{prt}(\mathbb{M}) \cup \{\mathbf{p}\}$
403 in T-ACT, and $\text{prt}(G) = \text{prt}(\mathbb{M})$ in T-GOAL. These make the invariant “the roles the protocol still
404 mentions are exactly the roles still running” hold at every node, ruling out a session that carries
405 a stray participant the protocol has forgotten (or vice versa) — the well-formedness that lets the
406 interleaving cases of Section D.3 match a bystander move on the session to a $\text{cap}(\cdot)$ -labelled move
407 on the type. T-END closes the discipline at the terminus: the terminated protocol End types a
408 single idle role $\mathbf{p}[0]$ — and, through the congruence $\mathbf{p}[0] \parallel \mathbb{M} \equiv \mathbb{M}$, any session all of whose
409 roles have terminated — the base case of the participant invariant, since $\text{prt}(\text{End}) = \emptyset$ and an idle
410 role contributes nothing to $\text{prt}(\mathbb{M})$. T-ACT is derivable only when $\langle W \rangle a \langle W' \rangle$ holds, which itself
411 presupposes $a \in \text{dom}(\Gamma)$: this is where hallucinated capabilities die, with T-MISS as the diagnostic.

412 C.3 The achievability judgment

$$\begin{array}{c}
\text{ACH} \\
\hline
\frac{\text{caps}(G) \subseteq \text{dom}(\Gamma) \quad \exists W_0 \models \text{init}. G \vdash (\mathbf{p}_1[S_{\mathbf{p}_1}] \parallel \dots \parallel \mathbf{p}_n[S_{\mathbf{p}_n}]; W_0) \wedge \Gamma; (G, W_0) \models \Diamond \varphi_{\text{goal}}}{\Gamma \vdash \{S_{\mathbf{p}}\}_{\mathbf{p}} : G \triangleright \Diamond \varphi_{\text{goal}}}
\end{array}$$

413 Read top-to-bottom: no hallucinated tools; some plausible initial world exists from which the de-
414 clared skills directly conform to the protocol (deadlock-free, every capability call guarded) *and* that
415 same world reaches the goal. The reachability half is unchanged from Section B.2 and decided on
416 Γ, G alone, before any $S_{\mathbf{p}}$ is known — exactly what lets the checker run on the LLM-compacted
417 pack the moment it exists; conformance is the additional, separately decidable check once the skills
418 are declared, and needs no transport lemma: the receiver-side subtyping slack is already inside T-
419 COMM.

420 C.4 Reachability rules

The diamond is derived by the following inductive system, realized by the checker as a finite forward search over $\rightsquigarrow$; unlike T-COMM/T-ACT/ T-GOAL, it never mentions a session $\mathbb{M}$, which is what

makes it decidable on the pack alone.

$$\begin{array}{c}
\text{REACH-GOAL} \\
\frac{W \models \varphi_{\text{goal}} \quad G = \text{End} \vee \exists G'. G = \checkmark \varphi_{\text{goal}}. G'}{\Gamma; (G, W) \models \Diamond \varphi_{\text{goal}}} \\
\\
\begin{array}{cc}
\text{REACH-STEP} & \text{REACH-OR} \\
\frac{(G, W) \rightsquigarrow (G', W') \quad \Gamma; (G', W') \models \Diamond \varphi_{\text{goal}}}{\Gamma; (G, W) \models \Diamond \varphi_{\text{goal}}} & \frac{\exists k \in I. \Gamma; (G_k, W) \models \Diamond \varphi_{\text{goal}}}{\Gamma; (\text{p} \rightarrow \text{q}; \{\ell_i. G_i\}_{i \in I}, W) \models \Diamond \varphi_{\text{goal}}}
\end{array}
\end{array}$$

D Metatheory

We separate what is *mechanized in Coq* (Theorems 1–3; Theorems 4–5 for the single-label communication fragment with full interleaving; the head-move action case; and the concrete instances, all axiom-free) from what is *proved on paper* (the action-interleaving cases, decidability, and the undecidability boundary).

D.1 Soundness of the abstraction

Let $\mathcal{C}$ be the concrete configuration space of protocol/world pairs, $\mathcal{A}$ the abstract configuration space the checker explores, and $\text{abs} : \mathcal{C} \rightarrow \mathcal{A}$ the abstraction. We require:

$$(\text{step-sim}) \quad C \rightsquigarrow C' \Rightarrow \text{abs}(C) \rightsquigarrow^* \text{abs}(C'), \quad (\text{goal-sim}) \quad \text{goal}_{\varphi_{\text{goal}}}(C) \Rightarrow \hat{\text{goal}}(\text{abs}(C)).$$

Lemma 1 (Transport). *If $C_0 \rightsquigarrow^* C$ then $\text{abs}(C_0) \rightsquigarrow^* \text{abs}(C)$.*

Proof. By induction on the reflexive-transitive closure. The empty run is reflexivity. For $C_0 \rightsquigarrow^* C' \rightsquigarrow C$, the induction hypothesis gives $\text{abs}(C_0) \rightsquigarrow^* \text{abs}(C')$ and STEP-SIM gives $\text{abs}(C') \rightsquigarrow^* \text{abs}(C)$; compose them. Mechanized as `reach_abs`, parameterized by STEP-SIM. $\square$

Lemma 2 (Reachability monotonicity). *If every step of $\rightsquigarrow_1$ is a step of $\rightsquigarrow_2$ (that is, $x \rightsquigarrow_1 y$ implies $x \rightsquigarrow_2 y$), then $s \rightsquigarrow_1^* w$ implies $s \rightsquigarrow_2^* w$.*

Proof. By induction on the closure $s \rightsquigarrow_1^* w$. The empty run transports by reflexivity. For $s \rightsquigarrow_1^* u \rightsquigarrow_1 w$, the induction hypothesis gives $s \rightsquigarrow_2^* u$; the hypothesis turns the last step $u \rightsquigarrow_1 w$ into $u \rightsquigarrow_2 w$; appending it gives $s \rightsquigarrow_2^* w$. Mechanized as `reach_mono`. $\square$

D.2 Refutation soundness, tolerance, monotonicity

Theorem 1 (Refutation soundness). *If the abstract system has no reachable goal state from $\text{abs}(G, W_0)$, then no declared run of that pack configuration reaches φ_{goal} . Hence an impossible verdict is never wrong relative to the pack, provided STEP-SIM and GOAL-SIM hold.*

Proof. We prove the contrapositive: if *some* declared run reaches the goal, then the abstract system reaches an abstract goal state. Suppose $C_0 \rightsquigarrow^* C$ with $\text{goal}_{\varphi_{\text{goal}}}(C)$. By Lemma 1, $\text{abs}(C_0) \rightsquigarrow^* \text{abs}(C)$, and by GOAL-SIM, $\hat{\text{goal}}(\text{abs}(C))$. Thus $\text{abs}(C)$ is an abstract goal state reachable from $\text{abs}(C_0)$, contradicting the hypothesis. Hence no declared run reaches φ_{goal} : a verdict of *impossible* — read off exactly the premise, that the abstract search exhausts without hitting an abstract goal — is never wrong. Mechanized, axiom-free as a theorem schema parameterized by the two simulation hypotheses (the existential witness is $\text{abs}(C)$): $\square$

```

449   Theorem refutation_sound : forall s0,
450     (~ exists a, reach_aste (abs s0) a /\ agoal a) ->
451     (~ exists w, reach_cstep s0 w /\ cgoal w).
452   (* Print Assumptions refutation_sound. => Closed under the global
453     context *)

```

Theorem 2 (Tolerance soundness). *If a coarser over-approximation (more edges) cannot reach the goal, neither can any finer system. Hence coarsening α to tolerate more detail never produces a false refutation.*

457 *Proof.* Let $\rightsquigarrow_{\text{fine}}$ and $\rightsquigarrow_{\text{coarse}}$ be the two abstract step relations, with the coarsening an *over-*
 458 approximation: every fine step is also a coarse step, $x \rightsquigarrow_{\text{fine}} y \Rightarrow x \rightsquigarrow_{\text{coarse}} y$. Suppose, for
 459 contradiction, that the finer system reaches the goal: $s_0 \rightsquigarrow_{\text{fine}}^* a$ with $\hat{goal}(a)$. By Lemma 2 (with
 460 $\rightsquigarrow_1 = \rightsquigarrow_{\text{fine}}$, $\rightsquigarrow_2 = \rightsquigarrow_{\text{coarse}}$), $s_0 \rightsquigarrow_{\text{coarse}}^* a$; and $\hat{goal}(a)$ still holds. So the coarser system reaches a
 461 goal state, contradicting the hypothesis. Hence no coarser-refuted goal is reachable in any finer sys-
 462 tem: coarsening α only *adds* abstract edges, so it can never turn an unreachable goal into a reachable
 463 one — refutation is preserved. Mechanized as `tolerance_sound`, axiom-free. $\square$

464 **Theorem 3** (Capability monotonicity). *If $\Gamma \subseteq \Gamma'$ and Γ reaches φ_{goal} , then Γ' reaches φ_{goal} .*
 465 *Granting tools never makes an achievable goal impossible.*

466 *Proof.* Enlarging the capability context only *adds* guarded effects: since $\Gamma \subseteq \Gamma'$, every capability fir-
 467 ing available under Γ is available under Γ' with the same guard and effect (WORLD-ACT quantifies
 468 over $a \in \text{dom}(\Gamma) \subseteq \text{dom}(\Gamma')$), so every step of the Γ -transition relation is a step of the Γ' -relation.
 469 Suppose Γ reaches the goal configuration C from C_0 . By Lemma 2, the same configuration is
 470 reachable under Γ' , and $goal_{\varphi_{\text{goal}}}(C)$ is unchanged because the goal predicate does not depend on
 471 Γ . Hence Γ' reaches φ_{goal} via the same witnessing run. The other premises of ACH are monotone
 472 in the same direction ($\text{caps}(G) \subseteq \text{dom}(\Gamma) \subseteq \text{dom}(\Gamma')$ still holds, and realizability and confor-
 473 mance do not mention Γ beyond $a \in \text{dom}(\Gamma)$), so the whole judgment is preserved. Mechanized as
 474 `cap_monotone`, axiom-free. $\square$

475 **Concrete instance.** The flight skill of Section 2, variant A, is mechanized as `FlightInstance`.
 476 We prove `flight_refuted` (no run reaches `booked` $\wedge$ `confirmation_sent`, since no step touches
 477 `conf`) and `booking_reachable` (a booking is still reachable). Both are axiom-free, witnessing
 478 non-vacuity of Theorem 1.

479 D.3 Operational correspondence: subject reduction and session fidelity

480 The direct judgment $G \vdash (\mathbb{M}; W)$ is not merely preserved by reduction: the session and its protocol
 481 step *in lockstep* over the labelled transition systems of Section B.2. This is the standard soundness
 482 backbone of a session calculus [1, 6], obtained here *directly*, with no projection and no merge. For
 483 world-changing interleavings, the statements below assume two explicit frame conditions: effects of
 484 participant-disjoint actions commute, and an action moved under a pending marker preserves that
 485 marker’s formula. Communication-only reductions satisfy both conditions vacuously.

486 **Lemma 3** (Residual capability). *If $(G, W) \rightsquigarrow_{\Lambda} (G', W')$, then $\Lambda \in \text{cap}(G)$.*

487 **Lemma 4** (Participant agreement). *If $G \vdash (\mathbb{M}; W)$, then $\text{prt}(G) = \text{prt}(\mathbb{M})$.*

488 Both follow by induction on the transition or typing derivation, respectively; they are the capability-
 489 membership and participant invariants used in the interleaving cases below.

490 **Theorem 4** (Subject reduction). *If $G \vdash (\mathbb{M}; W)$ and $(\mathbb{M}, W) \xrightarrow{\Lambda} (\mathbb{M}', W')$, then $(G, W) \rightsquigarrow_{\Lambda}$
 491 (G', W') for some G' with $G' \vdash (\mathbb{M}'; W')$.*

492 **Theorem 5** (Session fidelity). *If $G \vdash (\mathbb{M}; W)$ and $(G, W) \rightsquigarrow_{\Lambda} (G', W')$, then $(\mathbb{M}, W) \xrightarrow{\Lambda}$
 493 $(\mathbb{M}', W')$ for some $\mathbb{M}'$ with $G' \vdash (\mathbb{M}'; W')$.*

494 We prove both by induction on the typing derivation $G \vdash (\mathbb{M}; W)$, with a case analysis on the
 495 transition; the one auxiliary fact used throughout is that typing respects pointwise-equal sessions
 496 (`ctypes_ext`), which lets the interleaving cases reassociate independent updates without functional
 497 extensionality.

498 *Proof of Theorem 4 (subject reduction).* By induction on $G \vdash (\mathbb{M}; W)$.

499 *Case T-END* ($G = \text{End}$, every role idle). Then $\mathbb{M}$ has no output or pending action, so the only
 500 available transition is the goal observation S-GOAL at $\Lambda = \checkmark \varphi_{\text{goal}}$ when $W \models \varphi_{\text{goal}}$, which leaves
 501 $(\mathbb{M}, W)$ fixed; there is no residual type to preserve and the claim holds (otherwise no Λ -transition
 502 exists and it holds vacuously).

503 *Case T-GOAL* ($G = \checkmark \varphi.G_0$, with $W \models \varphi$ and $G_0 \vdash (\mathbb{M}; W)$, $\text{prt}(G_0) = \text{prt}(\mathbb{M})$). The transition
 504 $(\mathbb{M}, W) \rightarrow_{\Lambda} (\mathbb{M}', W')$ splits on its label.

505 • *Goal observation* ($\Lambda = \checkmark\varphi$, by S-GOAL, so $\mathbb{M}' = \mathbb{M}$ and $W' = W$). Since $W \models \varphi$, rule
 506 G-GOAL-E discharges the marker, $(\checkmark\varphi.G_0, W) \rightsquigarrow_{\checkmark\varphi} (G_0, W)$, and $G_0 \vdash (\mathbb{M}; W)$ is the
 507 premise: take $G' = G_0$.

508 • *Interleaving* ($\Lambda \neq \checkmark\varphi$). A goal marker is not a session construct, so this is already a
 509 transition of G_0 's session; the induction hypothesis gives $(G_0, W) \rightsquigarrow_{\Lambda} (G', W')$ with
 510 $\Lambda \in \text{cap}(G_0)$ and $G' \vdash (\mathbb{M}'; W')$. When the move is world-preserving (a communi-
 511 cation, $W' = W$), G-GOAL-I carries the still-pending marker along, $(\checkmark\varphi.G_0, W) \rightsquigarrow_{\Lambda}$
 512 $(\checkmark\varphi.G', W)$, and $\checkmark\varphi.G' \vdash (\mathbb{M}'; W)$ by T-GOAL (the side condition $\text{prt}(G') = \text{prt}(\mathbb{M}')$ is
 513 preserved because Λ acts on the same participants on both sides). When the move is *world-*
 514 *changing* (a capability firing, $W' \neq W$), G-GOAL-I still commutes it under the marker,
 515 $(\checkmark\varphi.G_0, W) \rightsquigarrow_{\Lambda} (\checkmark\varphi.G', W')$; re-typing the residual by T-GOAL then additionally asks
 516 $W' \models \varphi$. This follows from the goal-stability hypothesis on actions commuting under pend-
 517 ing markers. The global type sequences the marker before G_0 's actions whereas the session
 518 may fire the action first; the interleaving rule lets the type catch up under that hypothesis.
 519 Removing the hypothesis, rather than this conditional case, is the open correspondence
 520 problem recorded in Section 7.

521 *Case T-COMM*, with head $p \rightarrow q$ and branches G_i . Here $\mathbb{M}$ is $p[q! \dots] \parallel q[p? \dots] \parallel \mathbb{M}_0$, and each
 522 branch satisfies $G_i \vdash (p[P_i] \parallel q[Q_i] \parallel \mathbb{M}_0; W)$. If the transition is a goal observation $\Lambda = \checkmark\varphi$ (S-
 523 GOAL, leaving $(\mathbb{M}, W)$ fixed), it is matched by G-COMM-I: $\text{prt}(\checkmark\varphi) = \emptyset$ makes the disjointness
 524 premise trivial, and each branch realises the same observation by the induction hypothesis, using
 525 $\checkmark\varphi \in \text{cap}(G_i)$ — the observation concerns a marker the protocol actually pends (see the note
 526 below). Otherwise the transition is a communication $\Lambda = r \ell t$. Because p offers only outputs to q
 527 and q only inputs from p , the sender/receiver shapes force a dichotomy.

528 • *Head move* ($r = p$). Then $t = q$ and $\ell = \ell_k \in I$, and $\mathbb{M}' = p[P_k] \parallel q[Q_k] \parallel \mathbb{M}_0$.
 529 Rule G-COMM-E gives $(G, W) \rightsquigarrow_{p\ell_k q} (G_k, W)$, and $G_k \vdash (\mathbb{M}'; W)$ is exactly the k -th
 530 premise.

531 • *Interleaving move* ($r \neq p$). The shapes then force $r, t \notin \{p, q\}$: r is an output so $r \neq q$, and
 532 t is an input so $t \neq p$, while $t = q$ would force $r = p$. Since r, t lie in the bystanders $\mathbb{M}_0$,
 533 the *same* communication is enabled in each branch's configuration $p[P_i] \parallel q[Q_i] \parallel \mathbb{M}_0$
 534 (the head updates do not touch r, t). The induction hypothesis at each i yields G'_i with
 535 $(G_i, W) \rightsquigarrow_{\Lambda} (G'_i, W)$ and $G'_i \vdash (p[P_i] \parallel q[Q_i] \parallel \mathbb{M}'_0; W)$, where $\mathbb{M}'_0$ is $\mathbb{M}_0$ after the
 536 bystander step. As $\text{prt}(\Lambda) = \{r, t\}$ is disjoint from $\{p, q\}$, rule G-COMM-I assembles
 537 $(G, W) \rightsquigarrow_{\Lambda} (p \rightarrow q: \{\ell_i.G'_i\}, W)$, and re-applying T-COMM (the sender/receiver at p, q
 538 are untouched, and each branch is retyped by the induction hypothesis, reassociating the
 539 independent updates via `ctypes_ext`) gives the residual typing.

540 Communications leave the world fixed, so no independence hypothesis is needed. For a world-
 541 changing action at the head (T-ACT), G-ACT-E matches and the residual G is typed at the *same*
 542 post-effect world the rule reaches — this is exactly where the lockstep world pairing of the corrected
 543 T-ACT is used; a goal observation at an action-typed configuration is matched by G-ACT-I exactly
 544 as in the T-COMM case, since $\text{prt}(\checkmark\varphi) \cap \{p\} = \emptyset$ trivially. Interleaving *two* actions requires their
 545 effects to commute ($\text{eff}_a \circ \text{eff}_b = \text{eff}_b \circ \text{eff}_a$ for disjoint a, b), a frame-independence side condition that
 546 participant-disjointness alone does not supply; under it the T-ACT interleaving case goes through
 547 identically. $\square$

548 *Proof of Theorem 5 (session fidelity)*. By induction on $G \vdash (\mathbb{M}; W)$, inverting the global step
 549 $(G, W) \rightsquigarrow_{\Lambda} (G', W')$. T-END admits only the goal observation G-GOAL-E at $\checkmark\varphi_{\text{goal}}$, matched
 550 on the session by S-GOAL. For T-GOAL, the step is either G-GOAL-E — matched by S-GOAL
 551 on the session, the marker discharged — or G-GOAL-I, whose premise $(G_0, W) \rightsquigarrow_{\Lambda} (G'_0, W')$
 552 the induction hypothesis realises as an underlying session step that carries the marker along. For
 553 T-COMM, the global step is either G-COMM-E — and the prescribed head communication $p \ell_k q$ is
 554 enabled by S-COMM, landing in the k -th premise — or G-COMM-I with $\text{prt}(\Lambda) \cap \{p, q\} = \emptyset$; then
 555 the induction hypothesis on any branch produces a bystander communication, whose endpoints lie
 556 outside $\{p, q\}$, so S-COMM fires it on $\mathbb{M}$ and T-COMM retypes the result. The action cases mirror
 557 those of Theorem 4. $\square$

558 **Mechanization.** Both theorems are mechanized axiom-free for the single-label communication frag-
 559 ment — the fragment in which bystander interleaving is unconditional — in `DirectTypingSR.v`,
 560 over labelled session and global transition systems, with no projection, merge, or local type:

```

561   Theorem subject_reduction : forall W G s s' L,
562     ctypes W G s -> lstep L s s' ->
563     exists G', gstep W L G G' /\ ctypes W G' s'.
564   Theorem session_fidelity : forall W G G' s L,
565     ctypes W G s -> gstep W L G G' ->
566     exists s', lstep L s s' /\ ctypes W G' s'.
567   (* Print Assumptions => Closed under the global context (both). *)

```

568 The head-move case for world-changing *actions* is additionally mechanized in `DirectTyping.v`
 569 (`type_directed_safety`, also `progress`). Observable goal labels appear in neither Coq devel-
 570 opment: the mechanized label alphabet is communication-only, and its goal rule fuses marker dis-
 571 charge with a continuation step. Participant-matching side conditions, labelled action interleaving,
 572 and multi-branch choice are likewise absent. Completing a faithful mechanization of the paper rules
 573 remains work in Section 7.

574 **Concrete instance.** `HandoffInstance` mechanizes the planner/worker handoff of Section 1
 575 on both sides. The *good* pairing — planner sends, worker delivers, goal checked — is
 576 proved `ctypes`-typed (`good_typed`) and its run is proved to reach a world satisfying the goal
 577 (`good_runs_to_goal`). The *bad* pairing — both roles start with an input, exactly the deadlock
 578 of Section 1 — is proved `Stuck` (`bad_stuck`), and the contrapositive of `progress` then gives, for
 579 free, that *no non-trivial global type can ever type it* (`bad_untimeable`): the rule rejects the classical
 580 deadlocking handoff without any projection ever being computed.

581 D.4 Incompleteness, by design

582 **Proposition 1** (Incompleteness). $\Gamma; G \models \Diamond_{init} \varphi_{goal}$ may hold while no concrete runtime execution
 583 reaches φ_{goal} : the witnessing abstract path can be spurious when α collapses a distinction that
 584 mattered. The system is sound for $\times$ and incomplete for $\checkmark$.

585 This is the price of decidable, tolerant over-approximation; tightening α trades tolerance for preci-
 586 sion. The residue is confined to two edges, motivating the layered architecture:

- 587 • **Payload faithfulness** (bottom edge): a tool’s declared post-condition (“filters to fares <
 588 500”) may be false — a runtime obligation for a deterministic monitor (Layer C), not a
 589 static one.
- 590 • **Intent fidelity** (top edge): the formalized goal may not capture what the user meant —
 591 irreducibly semantic, surfaced at the single compaction checkpoint for human review.

592 D.5 Decidability and the autonomy boundary

593 **Theorem 6** (Decidability). *In the fragment with finitely many roles (static topology), regular global*
 594 *types represented by finite tail-recursive control graphs, decidable state logic $\mathcal{L}$, and $\mathcal{L}$ -definable*
 595 *effects, $\Gamma; G \models \Diamond_{init} \varphi_{goal}$ is decidable.*

596 *Proof.* The pack-level achievability judgment $\Gamma; G \models \Diamond_{init} \varphi_{goal}$ (Section B.2) is an existential
 597 reachability query over configurations (G', W) ; we exhibit a terminating decision procedure and
 598 argue its completeness.

599 *The search space is finite.* A configuration has two components. (i) *Control*, the residual global
 600 type G' reachable from G by $\rightsquigarrow$. Since G is a regular tree (tail-recursion: finitely many distinct
 601 subtrees, Section A.3) and each $\rightsquigarrow$ step either descends into a subtree (G-COMM-E, G-ACT-E) or
 602 erases a marker (G-GOAL-E), the set $\{G' : (G, _) \rightsquigarrow^* (G', _)\}$ is a subset of the finitely many
 603 subtrees of G , hence finite. (ii) *Symbolic world*. The boolean part is a valuation $B : \text{Pred} \rightarrow \mathbb{B}$
 604 over the finitely many predicates named in the pack, so there are $2^{|\text{Pred}|}$ boolean states. The numeric
 605 part is summarized by an accumulated constraint $\psi \in \mathcal{L}$ (a conjunction of the guards taken and
 606 the Nd -postconditions assumed); on a control-graph back edge the reference procedure applies the
 607 widening $\psi \mapsto \top$, forgetting accumulated numeric constraints. This one-element numeric summary

is an over-approximation and, together with the finite boolean valuations, leaves only finitely many symbolic worlds. The product with finite control is therefore finite.

Each edge is computable. Firing a capability requires checking $W \models pre_a$ and forming sp ; testing goal-satisfaction requires $W \models \varphi$. Both are entailment/satisfiability queries in quantifier-free linear integer arithmetic with uninterpreted predicates — a decidable theory, discharged by an SMT solver. Branch selection (REACH-OR) is a finite disjunction. Existence of an initial world is decided by the same SMT query over *init*; each satisfying symbolic initial valuation seeds the finite search.

Termination and correctness. Forward search (breadth-first over $\rightsquigarrow$) maintains a visited set of (control, symbolic-world) pairs; because that set is finite it adds each pair at most once, so the search halts. It answers ACHIEVABLE iff it enumerates a goal-satisfying configuration, and IMPOSSIBLE otherwise. Soundness of the IMPOSSIBLE verdict is Theorem 1; and widening only *enlarges* the reachable set (it weakens constraints, adding edges), so by Theorem 2 it never introduces a false refutation. Hence the procedure is a decision procedure for $\Gamma; G \models \Diamond_{init} \varphi_{goal}$. $\square$

Proposition 2 (Undecidability in a dynamic-topology extension). *Consider the extension G^+ of the grammar with spawn $r.G$, which creates fresh participants at run time. With an unbounded role population and dynamic channels, achievability in G^+ is undecidable.*

Proof sketch. By reduction from the control-state reachability problem for communicating finite-state machines (CFSMs), which is undecidable [10]. Fix a CFSM instance: finite-control machines M_1, M_2 exchanging messages over unbounded FIFO channels, and a target control state q_f of M_1 ; deciding whether a run reaches q_f is undecidable.

We compute, from this instance, a skill whose goal is achievable iff the CFSM reaches q_f . Each machine M_i becomes a role whose process mirrors its control graph: a transition that sends message m spawns, at run time, a fresh *courier* participant carrying m (this is exactly the dynamic-spawning capability the theorem hypothesizes), and a transition that receives m synchronizes with the oldest outstanding courier for that channel, so the unbounded family of couriers realizes the unbounded FIFO. The required order can be made explicit by assigning sequence numbers through the numeric world state and guarding receives on the next sequence number; the *Asg/Nd* machinery of Section A.2 supplies those updates. A boolean predicate at_{q_f} is established by the single capability guarding M_1 ’s entry into q_f , and we set $\varphi_{goal} = at_{q_f}$. By construction a run of the skill reaches a state satisfying φ_{goal} iff the CFSM has a run reaching q_f : the courier discipline then puts the skill’s reachable configurations in correspondence with the CFSM’s channel contents and control states. The construction is a computable map from CFSM instances to packs, so a decision procedure for achievability would decide CFSM control-state reachability — impossible. Hence achievability is undecidable once participants may be spawned dynamically. (The finiteness argument of Theorem 6 breaks at exactly the point this reduction exploits: an unbounded number of couriers makes the set of reachable controls infinite.) $\square$

The two results isolate one concrete boundary: static finite topology admits a total decision procedure, while unbounded dynamic spawning does not. This is a boundary on topology, not a claim that all forms of agent autonomy are undecidable. The implementation returns UNKNOWN when a pack violates the static-topology side condition of Theorem 6, unless an independent capability or establisher refutation has already proved it impossible.

D.6 The partition of obligations

Corollary 1. *The architecture separates “will the goal be achieved” into four obligations on disjoint substrates:*

Concern	Where	Mechanism
goal achievability	static, decidable	Coq specification; deterministic search; no LLM
per-step safety / least privilege	Layer B (future)	same engine, $\square$ + permission tiers
payload faithfulness	Layer C (future), run-time	deterministic monitors with blame
intent fidelity	top edge, synthesis	one human checkpoint

D.7 Verifiable feedback beyond a scalar reward

The verdict is a heterogeneous verification signal rather than a scalar score. An IMPOSSIBLE result identifies which premise failed and carries a blocking frontier; ACHIEVABLE carries a witness path and explicitly retains the two deferred obligations; UNKNOWN is an abstention outside the decidable fragment, not a refutation. These structured LLM outcomes can serve as cheap negative labels for automated skill or prompt search without an LLM judge, while the intent-fidelity checkpoint keeps the irreducible semantic decision with a human. We treat such search as an application of the verifier, not as a contribution evaluated here.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state that refutation is sound relative to the declared pack, distinguish UNKNOWN from refutation, and identify the mechanized and paper-only proof fragments.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The Limitations and Future Work section discusses relative soundness, static topology, abstraction coarseness, mechanization gaps, and the proof-of-concept corpus.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Appendix D states the simulation, stability, commutativity, finiteness, and topology assumptions with proofs or proof sketches; the artifact checks the explicitly identified Coq fragments.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 4–5 describe the deterministic checker, verdict semantics, corpus construction, binary accounting, and separate treatment of abstentions.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The anonymized supplemental artifact contains the checker, synthetic corpora, proof developments, pinned continuous-integration workflow, and reproduction commands.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [N/A]

700 Justification: The evaluation is a deterministic execution over declared packs; it performs
701 no training, optimization, sampling, or data splitting.

702 **7. Experiment statistical significance**

703 Question: Does the paper report error bars suitably and correctly defined or other appropri-
704 ate information about the statistical significance of the experiments?

705 Answer: [N/A]

706 Justification: Each corpus case and checker result is deterministic, so repeated trials have
707 no sampling variance; the paper reports exact counts rather than inferential statistics.

708 **8. Experiments compute resources**

709 Question: For each experiment, does the paper provide sufficient information on the com-
710 puter resources (type of compute workers, memory, time of execution) needed to reproduce
711 the experiments?

712 Answer: [Yes]

713 Justification: Section 5 states that evaluation is CPU-only, requires no model inference or
714 training, and gives the corpus size and solver timeout governing the run.

715 **9. Code of ethics**

716 Question: Does the research conducted in the paper conform, in every respect, with the
717 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

718 Answer: [Yes]

719 Justification: The work uses synthetic packs, no personal data or human subjects, and
720 makes bounded claims with explicit trust assumptions and limitations.

721 **10. Broader impacts**

722 Question: Does the paper discuss both potential positive societal impacts and negative
723 societal impacts of the work performed?

724 Answer: [Yes]

725 Justification: The Broader impact paragraph discusses reduced wasted execution as well as
726 false confidence from inaccurate packs and the human/runtime mitigations required.

727 **11. Safeguards**

728 Question: Does the paper describe safeguards that have been put in place for responsible
729 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
730 image generators, or scraped datasets)?

731 Answer: [N/A]

732 Justification: The artifact releases no trained model, scraped dataset, or other high-risk
733 asset; it contains a deterministic checker and synthetic examples.

734 **12. Licenses for existing assets**

735 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
736 the paper, properly credited and are the license and terms of use explicitly mentioned and
737 properly respected?

738 Answer: [N/A]

739 Justification: The evaluation does not reuse an external dataset or model; prior methods
740 and software foundations are credited through citations.

741 **13. New assets**

742 Question: Are new assets introduced in the paper well documented and is the documenta-
743 tion provided alongside the assets?

744 Answer: [Yes]

745 Justification: The supplemental artifact documents the checker interface, pack schema, syn-
746 thetic corpora, proof scope, limitations, and reproduction workflow.

747 **14. Crowdsourcing and research with human subjects**

748 Question: For crowdsourcing experiments and research with human subjects, does the pa-
 749 per include the full text of instructions given to participants and screenshots, if applicable,
 750 as well as details about compensation (if any)?

751 Answer: [N/A]

752 Justification: The research involves neither crowdsourcing nor human-subject experiments.

753 **15. Institutional review board (IRB) approvals or equivalent for research with human**
 754 **subjects**

755 Question: Does the paper describe potential risks incurred by study participants, whether
 756 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
 757 approvals (or an equivalent approval/review based on the requirements of your country or
 758 institution) were obtained?

759 Answer: [N/A]

760 Justification: The research involves no study participants or human-subject data.

761 **16. Declaration of LLM usage**

762 Question: Does the paper describe the usage of LLMs if it is an important, original, or
 763 non-standard component of the core methods in this research? Note that if the LLM is used
 764 only for writing, editing, or formatting purposes and does not impact the core methodology,
 765 scientific rigor, or originality of the research, declaration is not required.

766 Answer: [Yes]

767 Justification: Sections 1 and 4 describe LLM compaction and bounded repair, explicitly
 768 place them outside the trusted checker, and state that the reported corpus evaluation does
 769 not depend on live model inference.